

MARS: A Memory-Augmented Rust Shell that Learns From Your Commands

Anjishnu Kumar*

Microsoft Copilot AI

Redmond, WA

anjikumar@microsoft.com

Abstract

This paper presents MARS, a Memory-Augmented Rust Shell: a terminal editor, multiplexer, and LLM agent that ship as a single binary. The assistant failures that matter most at a terminal are information gaps, not capability gaps: “run the unit tests” is `cargo test` in this project and `npm test` in the next, and no amount of model scale can recover which one the user means. MARS closes the gap by living where the information lives. It captures the commands the user actually accepts, retrieves them with plain BM25 over local files (no embeddings, no data leaving the machine), and injects them into the prompt. Under a leakage-controlled protocol in which the store holds only *paraphrases* of earlier requests, this lifts a small model (claude-haiku-4-5) from 38% (9/24) to 96% (23/24) on project-specific commands, one short of the verbatim-lookup ceiling: generalization across a paraphrase gap, not lookup of a planted answer. Pointed at the tool’s own documentation, the same mechanism raises accuracy on questions about the tool from 43% to 76% overall, with the decisive gain on plain-English reconfiguration (17% to 93%); the residual failures are informative, arising exactly where a strong parametric prior overrides retrieved specifics. We describe the registry-grounded directive protocol, the corrective-memory loop, the architecture that keeps the agent present across machines while the API key stays home, and a self-instrumenting evaluation built from the agent’s own debug log. MARS is open source (MIT); a screencast accompanies this paper.

1 Introduction

LLM assistants have become a routine part of terminal work. A typical user, an ML researcher babysitting training runs across a laptop and a handful of remote GPU machines, asks an assistant to explain a CUDA out-of-memory trace or

draft an `rsync` command, and the assistant is the weakest link, for two reasons that compound. It is generic: it runs in a separate window and sees only the fragment the user pastes into it. It is also amnesiac: it forgets the command the user corrected yesterday, forgets “where was I” on reattach, and knows nothing about the editor it sits beside. In the remote setting that defines GPU work there is a third problem: reaching the work at all usually means exporting an API key onto a machine the user does not own, where it lingers in the environment and the shell history.

Our observation is that these failures are information gaps, not capability gaps. A project-specific command is unknowable from the request text: no model, at any scale, can know that this repository launches training with a particular `sbatch` invocation settled on last week. The missing information exists in exactly one place, the user’s own terminal, in two corpora a browser-tab assistant never keeps: the history of commands the user actually ran, and the tool’s own documentation, action registry, and configuration schema. Neither corpus is large; what they require is an agent that lives beside them, persists between requests, and captures corrections as they happen. Per-invocation CLI agents (Anthropic, 2025) live in the shell but do not automatically capture the corrections a user makes (§2). We built MARS to do all three¹; it hosts the user’s shell in its panes rather than replacing it.

Framing the gap as informational says precisely what closing it requires: retrieving the one exemplar that carries the missing fact, and nothing more. We therefore hold the retriever fixed at plain BM25 over a flat file, as a deliberate control: any gap it closes is attributable to the informa-

* Work done before joining Microsoft.

¹Code, gold sets, and frozen result files: <https://github.com/anjishnu/mars> (MIT). Downloadable package: <https://crates.io/crates/mars-terminal>. Screencast: [URL].

tion, not to retrieval machinery. Under a leakage-controlled protocol in which the store holds only *paraphrases* of the user’s earlier requests, retrieval lifts claude-haiku-4-5 from 38% (9/24) to 96% (23/24) on project-specific commands, one item short of the verbatim-lookup ceiling: generalization, not lookup. In a domain this constrained, memory does not need to be sophisticated to be decisive; it needs to be present.

This paper makes four contributions. First, an open-source, single-binary terminal agent (§3) whose memory never leaves the machine: a registry-validated directive protocol, a corrective-memory loop, and a model cascade. Second, two axes of *local* memory (§3.4–3.5): the user’s own commands, which close the personalization gap, and the tool’s own ground truth, which grounds self-knowledge and reconfiguration. Third, a leakage-controlled protocol for evaluating agent memory (§5): the store holds independently generated paraphrases, the test uses the original requests, and a verbatim condition bounds the lookup ceiling. Fourth, the architecture that makes local memory always present (§4): a persistent daemon, and an SSH broker that keeps the API key off remote machines. MARS is about 9,500 lines of Rust (Kumar, 2026).

MARS is private by construction: the memory stores are flat local files the user can read, edit, and erase; retrieval is lexical, so nothing embedded ever leaves the machine; the only network call is the completion itself, and even that can stay local with a local model (§3.2); on remote machines the key never leaves home (§4); and every line that reaches a prompt first passes a redaction filter (Ethics Statement).

2 Related Work

Terminal tooling. Multiplexers such as tmux (Marriott and tmux contributors, 2024) and Zellij (Zellij contributors, 2024) provide persistence but no intelligence. Atuin (Huxtable and Atuin contributors, 2024) and thefuck (Iakovlev and contributors, 2024) share our premise that the user’s own history is valuable, but neither feeds it to a model. AI terminals such as Warp (Warp, 2024) add command search but see only the commands run inside them. CLI agents such as Claude Code (Anthropic, 2025) and GitHub Copilot in the CLI (GitHub, 2024) put a cloud model in the shell and can read the same local files MARS reads; how-

ever, at the time of writing their persistent context is curated by hand (project memory files) rather than captured automatically from the commands a user accepts, and every remote machine needs its own credential; IDE agents (Anysphere, 2024) lose context when the window closes. MARS differs in the combination: one persistent process that hosts the editor, captures every accepted correction, grounds the agent in its own action registry, and keeps the credential home.

Agents, retrieval, and NL-to-commands.

NL2Bash (Lin et al., 2018) and Tellina (Lin et al., 2017) established the NL-to-shell task and its corpora; we build on the task rather than the models, personalizing translation with the user’s own history. Directive-style acting relates to ReAct (Yao et al., 2023) and tool use to Toolformer (Schick et al., 2023), though we use a portable text protocol with registry grounding rather than provider-native tool calls. Our memory is retrieval-augmented generation (Lewis et al., 2020) in its simplest lexical form; the novelty is the setting: retrieval over the user’s *own* local context, embedded where it is produced. The residual failures we observe on Axis B, where a strong parametric prior overrides a retrieved specific (§5.4), are a deployed instance of the context-versus-parametric knowledge conflicts studied in question answering (Longpre et al., 2021); the personalization axis relates to benchmarks such as LaMP (Salemi et al., 2024), though our memory is captured from use rather than provided as a profile.

Computer-use agents. A parallel line of work drives whole desktops rather than terminals: Anthropic’s computer use (Anthropic, 2024) operates GUIs from screenshots, OSWorld (Xie et al., 2024) benchmarks such agents, OS-Atlas (Wu et al., 2025) pretrains a GUI-grounding action model, and Plan-and-Act (Erdogan et al., 2025) separates planning from execution. The terminal offers those ambitions a simpler substrate: the action space is already text, so grounding is a registry lookup rather than a vision problem, and a plan is a command sequence the user can read.

3 The Memory-Augmented Agent

3.1 Design considerations

The agent should be another input device, not a second system — not a subsystem bolted beside

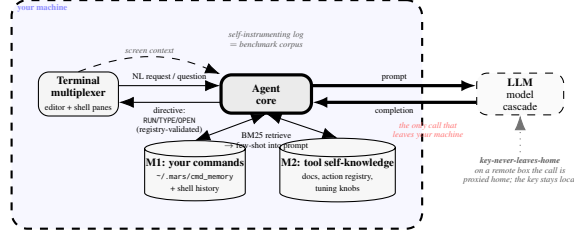


Figure 1: MARS retrieves with lexical BM25 from two local stores (M1, the user’s past commands; M2, the tool’s own docs and registry) and injects the results into the prompt. Only the model call leaves the machine; even the key stays home.

the editor with its own command language, state, and failure modes. Everything MARS can do is a variant of a single Action enum, and three interfaces resolve a user’s intent to it: a direct key-chord, a fuzzy command bar, and the agent (natural language). All three terminate in a single dispatch point, so a capability added once is chord-bindable, searchable, and agent-invokable. Figure 1 sketches the system and its retrieval loop.

This choice trades expressive freedom for grounding: the agent can only name actions that exist in the registry and retrieves from flat local files, so the surface a user must trust is small and easy to audit, but MARS cannot invent a capability at runtime, and lexical retrieval will miss paraphrases an embedding model would catch (§5). In the terminal domain we believe this trade is strongly favorable.

3.2 The agent as a validated input device

An agent that acts on the user’s workspace must be no less safe than a keypress, never a bypass around the editor’s gates. Following the mode-error tradition in interface design (Norman, 1988), we treat the agent as one input device among several, behind the same confirmation gates as a keystroke. The implementation is a trailing-line *directive* protocol: the model receives a live catalog of every action, description, and keybinding, and ends its reply with at most one directive, each gated by confirmation — RUN: <Action> fires a registry action, TYPE: <cmd> types a command into a shell pane, OPEN: path:line jumps to a file location, and NEED: <context> pulls more context. We chose plain text over provider-native tool-calling APIs because it works identically across OpenAI-compatible endpoints, a native Anthropic path, and local open models (which also keep prompts on the machine; see Ethics Statement), and renders as human-readable text the user confirms be-

fore anything fires; the cost is the stricter typing a schema provides.

The grounding comes from the registry check: a RUN: whose name does not resolve against the live Action set is rejected before dispatch, so the model cannot execute a hallucinated capability, and a destructive action passes the same confirmation gate a human would. The NEED: directive closes a different gap: rather than receiving the full screen with every request, the model can ask for more context (a pane’s scrollbar, another tab); MARS supplies it and re-asks exactly once, so no unbounded retrieval loop can form.

3.3 Natural-language shell translation

The bread-and-butter agentic request at a terminal is to turn English into the right command. *Translation should be assistive and reversible, never an automatic execution.* The implementation lives on the command bar: the user presses C-Space (we write keybindings in Emacs notation: C- is Ctrl, M- is Meta) and types plain English; MARS sends the request with on-screen context (working directory, recent output) to the model cascade (§3.7) and renders the proposed command at the cursor to run, edit, or discard. The same translator serves as the command bar’s fallback (§4). Translation by itself is a commodity capability (Lin et al., 2018, 2017); what personalizes it is the memory we describe next.

3.4 Memory 1: the user’s own commands

The user’s own accepted commands are the highest-value context available, and they are local to the terminal by construction. Asked to “launch the training run,” a generic model produces a plausible python train.py when this project in fact uses sbatch --gres=gpu:8 scripts/train.slurm. The implementation is a corrective loop. When the user accepts (or edits and runs) a translated command, MARS appends

the (request, command) pair to a local JSON-lines file; it also reads recent shell history from \$HISTFILE. On the next request, MARS ranks these entries against the query with BM25 (Robertson and Zaragoza, 2009), with standard parameters ($k_1=1.5$, $b=0.75$) and no index to build, and injects the top three pairs as few-shot exemplars, together with up to three ranked lines of recent shell history.

The loop is corrective in the precise sense that a command the model got wrong once becomes one it gets right thereafter, because the user’s correction is what enters the store; a per-invocation assistant never observes the correction. The retriever is deliberately unremarkable; the finding is that in this domain unremarkable retrieval is decisive (§5).

3.5 Memory 2: the tool’s own documentation

An embedded agent should answer questions about its tool from ground truth rather than a guess, and a registry generated from the code is ground truth that cannot name a capability that does not exist. MARS’s action registry, keybindings, and tuning knobs each carry a human-written description, so documentation is generated from the code itself. The same BM25 retriever runs over this second corpus (documentation chunked into passages, plus the live registry and described knobs); on a NEED: docs pull, MARS injects the top passages, so “how do I split the screen?” is answered with the real keybinding and “make the accent orange” resolves to the actual knob and a confirmable edit. When retrieval returns nothing, the agent reports that it cannot find such a capability rather than inventing one.

3.6 Simplicity as a control

The instinct when building a memory system is to reach for embeddings and a vector database; we keep the retriever deliberately trivial as a control, so any gap it closes is attributable to the information, not the machinery. The terminal also favors lexical retrieval: command vocabulary is small, repetitive, and lexically stable, so a query and its useful memory tend to share surface tokens (the regime where BM25 is strong and dense retrieval’s paraphrase advantage is marginal); the corpora are small; and a single relevant exemplar supplies the exact string the user wants reproduced. We therefore treat embeddings as a cost to be deferred until the domain broadens (§6). The empirical support

is §5, in which BM25 bridges paraphrases whose mean word-Jaccard similarity to the query is only 0.36, with nothing more than a prompt, a local file, and a standard ranking function.

3.7 The model cascade

Model choice should be right-sized per task. Most LLM traffic at a terminal is cheap and can be served by small or free models, but a single fixed model either wastes capability on easy tasks or fails hard ones, and hits rate limits where volume is high. We distinguish two orthogonal reasons to change models. Rotating *for limits* moves a task horizontally on a rate-limit error; because each provider meters usage on its own counter, rotating across N families multiplies the effective free ceiling by roughly N . Escalating *for quality* moves a task vertically to a stronger tier, and only on a detected failure. Detection is free for one class of failures: because the agent’s output is structured, a RUN: that fails the registry check is an unambiguous signal that a small model erred, and it triggers one retry at a higher tier. The benefit is headroom against the provider quota walls we later hit during evaluation (§5.1).

4 System: The Substrate for Local Memory

Local memory is only useful if the agent is reliably present where the memory lives, across disconnects and on machines the user does not own; the architecture guarantees that presence.

One binary, one core. MARS is a single Rust binary whose application core owns all state and never touches a TTY: input arrives as source-neutral events, and rendering is a stateless projection onto ratatui (a Rust terminal-UI library). The same core runs unmodified as a standalone terminal, a headless daemon, and a test backend, so the evaluation drives the real system with no mocks (§5).

A persistent daemon. We implemented persistence as a process split rather than as a save-and-restore feature: a server runs the entire application headless and emits the editor’s own ANSI byte stream over a Unix socket; a thin client owns the real terminal and pumps bytes. The benefit for memory is direct: because the core keeps running with no client attached, the command store and the user’s session survive a closed laptop, and an

overnight run can be watched and, on request, summarized for the user’s return.

Classifier-free routing. The command bar routes on a *sigil* rather than a learned classifier: the first character of the input names the target namespace. `!` runs a shell command, `?` asks the agent, `@` opens the file navigator, and plain text fuzzy-searches the action registry. Plain text that matches nothing in the registry falls back to the translator of §3.3, whose proposal the user confirms before it runs. The fallback stays deterministic, triggered by a registry miss rather than a classifier score, so there is no routing model to train or explain.

The key never leaves home. *A secret is a fact about the user, not the machine, and should live in one place.* The implementation is a broker. A home-side daemon (`mars keyd`) holds the key; when the user runs `mars ssh`, MARS reverse-forwards the broker’s socket over the SSH connection, and the agent on the remote box makes its model calls by proxying each request home, where the broker injects the credential and streams the completion back. The key never lands on the remote machine at all, so a compromised box has nothing to exfiltrate; the remaining exposure is the open tunnel, which a compromised box can use but not steal from, and which closes with the connection.

The log is the benchmark. Running with `--llm-debug` appends one JSON line per model call (task, model, token counts, latency, full prompt and reply) to a local stream, and records whether each suggestion was accepted, edited, or rejected. The log is both a cost profile the user can inspect and, as §5 shows, the source of every prompt, context, and token count the evaluation reports: the system instruments itself.

Demonstration. The demo walks attendees through the loop the paper measures, live: install MARS with one cargo command; reject a wrong translation, correct it, then issue a paraphrase of the same request and watch the corrected command return from memory; ask “how do I split the screen?” and retune the accent color in plain English, both answered from retrieved documentation; detach and reattach a running session; and open `mars ssh` with the key staying on the presenter’s laptop, the debug log streaming every call in a side pane. The screencast follows the same script.

5 Evaluation

5.1 Setup

We evaluate the two memory axes on frozen, a priori, author-written gold sets and a single small model under test: 112 command-translation items (88 general, 24 project-specific) and 82 self-knowledge question–answer items (52 knowledge, 30 reconfiguration); Axis A exercises Memory 1 (§3.4), Axis B Memory 2 (§3.5). The model under test is `claude-haiku-4-5`: the thesis requires a *small* model, so that personalization has a gap to close, but not necessarily a free one. Two free tiers (Qwen3-32B on Groq, Gemini Flash-Lite) exhausted their daily quotas mid-batch (the wall the cascade of §3.7 routes around), so we ran the final numbers on a small paid model; all reported runs had zero empty completions.

Correctness is scored by a stronger judge model, `claude-sonnet-5`, following LLM-as-judge practice (Zheng et al., 2023), so that the model under test does not grade itself; judge and testee share a model family (see Limitations).² The corrective-memory store is seeded per condition from paraphrases generated by a separate LLM pass and frozen alongside the gold sets; it contains the 24 seeded pairs and no distractors, and retrieval injects the top three (§3.4). Prompts, contexts, and token counts come from the self-instrumenting log of §4.

5.2 Axis A: corrective memory closes an information gap

The base numbers in Table 1 confirm the premise: the small model is a competent general shell author at 90% but falls to 42% on project-specific commands it cannot know. These are missing-knowledge failures, chiefly *ecosystem blindness* (Appendix A), exactly what local memory targets.

The corrective-memory experiment isolates that fix under the leakage-controlled protocol (stored paraphrases at mean word-level Jaccard 0.36 to the test query, tested on the original requests). Cold accuracy is 38% (9/24); paraphrase memory raises it to 96% (23/24), with fifteen items flipping to correct and one regressing from correct to wrong (the appendix examines the flips and the regression); verbatim memory reaches the lookup ceiling of

²Both models ran at provider-default sampling (Claude 4.5-series models reject the temperature parameter); prompt templates ship with the source (`src/prompts/`), alongside the gold sets and frozen result files.

Setting	Accuracy
General ($n=88$)	90% (79/88)
Personalized ($n=24$)	42% (10/24)
Overall ($n=112$)	79% (89/112)
<i>Corrective memory ($n=24$, leakage-controlled)</i>	
Cold (no memory)	38% (9/24)
Paraphrase memory	96% (23/24)
Verbatim memory (ceiling)	100% (24/24)

Table 1: Command translation (claude-haiku-4-5, judged by claude-sonnet-5). The corrective-memory block is leakage-controlled (§5.2): 96% measures generalization across a paraphrase gap; verbatim is the lookup ceiling. Base and cold come from separate runs that disagree on five items in both directions, netting to one (Appendix A).

Question type	No memory	+ Docs memory
Knowledge (how-do-I)	58% (30/52)	65% (34/52)
Reconfigure	17% (5/30)	93% (28/30)
Overall ($n=82$)	43% (35/82)	76% (62/82)

Table 2: Self-knowledge and self-reconfiguration (same models). Docs memory is decisive on reconfiguration, where retrieved knob descriptions supply the exact file, name, and default. The docs condition also adds an explain-don’t-act instruction; Appendix A separates its effect from that of the retrieved identifiers.

100% (24/24): simple BM25 over a local file recovers almost all of the personalization gap by generalizing, not by lookup. Memory adds about 66 tokens per call; the sign of that cost is model-dependent: on a reasoning model, the same exemplars suppress chain-of-thought and net cost falls (Appendix A).

5.3 Axis B: docs memory grounds identifiers, not priors

Retrieved documentation lifts accuracy on questions about MARS from 43% to 76% (Table 2), but the gain is unevenly distributed. On “how-do-I” knowledge questions it is modest (58% to 65%): the no-memory model already answers many correctly, and even with docs it sometimes hallucinates a keybinding the docs state exactly (Appendix A); §5.4 gives this asymmetry a shape. The decisive gain is on *reconfiguration* (turning a request like “autosave more often” into a concrete knob edit), which rises from 17% to 93%. Without the docs the model invents config filenames; with the knob descriptions retrieved, it emits the exact knob = value in `tuning.json`. One caveat: the docs condition also adds an explain-don’t-act

instruction, which repairs answers that fired a directive instead of explaining; the identifier-level repair requires the retrieved text (Appendix A).

5.4 Discussion: when retrieval beats the prior

Read together, the two axes locate the boundary of lexical memory. Retrieval is decisive when the missing piece is an identifier the model can copy intact: a project’s cargo test, a `--config` path, a knob name carrying its file and default. It is weakest when the model holds a strong parametric prior over the same slot: keybindings gain little because a plausible chord such as C-f overwrites the retrieved C-s, and the experiment’s single regression (p19) is the same conflict in miniature, a Rust-shaped ecosystem prior overriding a weakly retrieved Python exemplar. This is a naturally occurring instance of the context-versus-parametric knowledge conflicts studied in question answering (Longpre et al., 2021), observed here in a deployed agent rather than a constructed benchmark. It yields a design rule: place strings the model must not paraphrase (flags, paths, identifiers) into memory verbatim, and expect diminishing returns wherever the answer competes with a common default; the next unit of complexity is recency-and-project weighting on the retrieved side of these conflicts (§6).

6 Future Work

Four directions follow from the substrate: mining repeated command sequences into editable procedures, treating skill synthesis as the plan-then-execute decomposition of long-horizon agents (Erdogan et al., 2025); recognizing the shape of work above the text of commands, a training run or a debug session, and flagging deviation (Wu et al., 2025); semantic retrieval and a broker-carried store (§4) once mined skills widen the paraphrase gap BM25 bridges, with the redaction pass (Ethics Statement) growing toward learned secret detection; and learned routing from the log’s per-task accept/reject record, letting the watch capability summarize an overnight run into memory unprompted.

7 Conclusion

We described MARS, a single-binary terminal editor, multiplexer, and agent with memory over the user’s own context; plain lexical retrieval nearly closed the personalization gap and substantially

narrowed the self-knowledge gap. The through-line of both system and agenda is ownership: local, private, compounding memory. A cloud assistant starts every conversation a stranger; an agent that keeps its user’s memory becomes a little less generic and a little less amnesiac with every correction. The same substrate, enriched, points to agents that learn your skills and understand your work.

Limitations

Five limitations bound these results. First, the gold sets, though frozen before any run, are modest (112 and 82 items), so the personalized results should be read as evidence that memory closes this gap on this set, not as a claim at scale. Second, only one model is under test, so the numbers characterize the small, inexpensive regime MARS targets rather than models broadly; whether paraphrase generalization holds for a larger or reasoning model is an open question. Third, the judge (claude-sonnet-5) is stronger than the model under test but comes from the same model family, so a family-level preference cannot be ruled out, and an LLM judge carries noise in any case; the appendix quantifies the observed slack at a few points in either direction. Fourth, the corrective-memory store contained only the 24 seeded pairs during the experiment. A long-lived store holds hundreds of entries, and although the terminal domain’s lexical stability favors BM25 (§3.6), we have not measured retrieval precision against a realistic population of distractors. Fifth, the evaluation is offline against frozen gold sets; no longitudinal or in-the-wild usage study is reported. The scoring could also be tightened without touching the system: placeholder-normalized scoring and distinct marking of honest abstentions would remove a band of judge slack the appendix quantifies.

Ethics Statement

MARS reads the user’s shell history (\$HISTFILE) and stores accepted commands in a plaintext local file (~/.mars/cmd_memory). Shell history can contain secrets: tokens pasted into commands, passwords embedded in URLs, internal hostnames. Retrieval could therefore inject such strings into a prompt bound for a cloud provider. Every memory and history line runs through a redaction filter before prompt injection (known cre-

dential prefixes, password=/token:-style values, URL credentials, plus a user-editable denylist at ~/.mars/denylist); the store itself stays intact and user-editable, in-editor actions expose, edit, and erase it, and users who work with sensitive history can point MARS at a local model, which keeps prompts on the machine entirely. Redaction is heuristic and cannot catch every secret shape. On the credential side, the key broker (§4) is itself a mitigation: it removes the incentive to scatter API keys across remote machines. The evaluation involved no human subjects and no data beyond author-written gold sets and the authors’ own usage logs.

References

- Anthropic. 2024. Introducing computer use. <https://www.anthropic.com/news/3-5-models-and-computer-use>. Accessed July 2026.
- Anthropic. 2025. Claude code. <https://www.anthropic.com/claude-code>. Accessed July 2026.
- Anysphere. 2024. Cursor: The AI code editor. <https://cursor.com>. Accessed July 2026.
- Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. 2025. Plan-and-act: Improving planning of agents for long-horizon tasks. arXiv:2503.09572.
- GitHub. 2024. GitHub Copilot in the CLI. <https://docs.github.com/en/copilot/github-copilot-in-the-cli>. Accessed July 2026.
- Ellie Huxtable and Atuin contributors. 2024. Atuin: Magical shell history. <https://atuin.sh>. Accessed July 2026.
- Vladimir Iakovlev and contributors. 2024. The fuck: Corrects your previous console command. <https://github.com/nvbn/thefuck>. Accessed July 2026.
- Anjishnu Kumar. 2026. MARS: A memory-augmented agentic terminal multiplexer. <https://crates.io/crates/mars-terminal>. Software, installable via cargo install mars-terminal. Accessed July 2026.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

- Xi Victoria Lin, Chenglong Wang, Deric Pang, Kevin Vu, Luke Zettlemoyer, and Michael D. Ernst. 2017. Program synthesis from natural language using recurrent neural networks. *University of Washington Department of Computer Science and Engineering, Tech. Rep. UW-CSE-17-03-01*.
- Xi Victoria Lin, Chenglong Wang, Luke Zettlemoyer, and Michael D. Ernst. 2018. NL2Bash: A corpus and semantic parser for natural language interface to the linux operating system. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*.
- Shayne Longpre, Kartik Perisetla, Anthony Chen, Nikhil Ramesh, Chris DuBois, and Sameer Singh. 2021. [Entity-based knowledge conflicts in question answering](#). In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Nicholas Marriott and tmux contributors. 2024. tmux: a terminal multiplexer. <https://github.com/tmux/tmux>. Accessed July 2026.
- Donald A. Norman. 1988. *The Psychology of Everyday Things*. Basic Books.
- Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.
- Alireza Salemi, Sheshera Mysore, Michael Bendersky, and Hamed Zamani. 2024. [LaMP: When large language models meet personalization](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7370–7392.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Warp. 2024. Warp: The intelligent terminal. <https://www.warp.dev>. Accessed July 2026.
- Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. 2025. OS-ATLAS: A foundation action model for generalist GUI agents. In *International Conference on Learning Representations (ICLR)*.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. 2024. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.
- Zellij contributors. 2024. Zellij: a terminal workspace and multiplexer. <https://zellij.dev>. Accessed July 2026.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

A Qualitative Examples and Error Analysis

Every example in this appendix is quoted verbatim from the frozen result files the evaluation was scored on: `axis_a.jsonl` (112 translation items), `lift.jsonl` (24 leakage-controlled cold/paraphrase/verbatim items), and `axis_b.jsonl` (self-knowledge, 164 rows over the two memory conditions), with the model under test `claude-haiku-4-5` and the judge `claude-sonnet-5` throughout. The purpose is to show, from the raw data, exactly where a simple local-memory mechanism helps, where it does not, and where the measurement itself is the limiting factor.

A.1 Axis A: NL→shell translation

On general commands the small model is a competent shell author (79 of 88 correct), and its correct outputs are not only the easy one-liners: it composes multi-stage pipelines with the right utility at each stage. Judged correct, for example:

```
a20 find the ten largest files under the cwd
-> find . -type f -printf '%s %p\n' \
    | sort -rn | head -10
```

This is the premise of the paper made concrete: general shell translation is a near-solved commodity even on a small, inexpensive model, so the accuracy that matters is the personalized slice, which the model cannot know from the request text alone.

Two error classes account for the general-command failures (this run produced zero empty completions, so every failure is a real answer). The first is *placeholder and precision misses*: a stand-in the gold set spells differently (a69 `cat -n`

Request	Cold (no memory)	+ Paraphrase memory
run the unit tests	npm test	cargo test
lint the code	eslint .	ruff check .
build the release binary	make release	cargo build --release
install project dependencies	npm install	uv sync
check code formatting	shellcheck *.sh	black --check .
clean the build artifacts	rm -rf build/ dist/ ...	cargo clean
run the training job	python train.py	python train.py --config configs/base.yaml
deploy the model	(asks for more context)	./scripts/deploy.sh --prod

Table 3: Representative wrong/partial→correct flips from the leakage-controlled experiment (lift.jsonl). The store held only a *paraphrase* of each prior request (mean word-Jaccard 0.36) and the model was tested on the original, so a warm win requires generalization across the paraphrase gap, not lookup. The top block shows *ecosystem correction* (a Rust/uv project answered with npm/eslint/shellcheck defaults); the last two rows show *exact-argument recall* and *abstention rescue*. All warm outputs match gold.

vs. gold `cat -n file.txt`) or a plausible but semantically different tool (a38 “show my current IP” → `ip addr show`, a LAN address, where the reference wants `curl ifconfig.me`). The second dominates the personalized slice: *ecosystem blindness*, answering a Rust or uv project with the statistically commonest ecosystem, as in p04 “run the unit tests” → `npm test` (gold `cargo test`); Table 3 quotes more. A milder pattern is *honest abstention*: on the hardest project commands the model declines rather than bluff, safe behavior that still scores wrong against a gold command.

The judge assigns a *partial* band that captures near-misses, and its slack explains why the two no-memory measurements of the 24 personalized items disagree: the base row of Table 1 (42%, 10/24) and the lift experiment’s cold condition (38%, 9/24) come from separate runs, and they differ on five items. Three differ because the cold completions were regenerated and the model produced a different command (p10, p14, p17); two because the judge graded the identical output differently across runs (p01 source `venv/bin/activate`, correct vs. partial; p22 `docker push`, wrong vs. correct). The disagreements move in both directions and net to one item. Every point estimate in this paper therefore carries a couple of points of generation-and-judging slack, and we read the results through that band.

A.2 Corrective memory

Under the leakage-controlled protocol, with only a *paraphrase* of each prior request in the store (mean word-Jaccard 0.36 to the test query), 23 of 24 project-specific items are correct, up from 9 correct cold: 15 items flip to correct and one regresses

(below); Table 3 quotes representative flips verbatim. Two sub-mechanisms are visible: *ecosystem correction* (the commonest default corrected to the user’s real toolchain) and *exact-argument recall* (the right utility gains the precise config, path, or split the user expects). Of the 15 non-correct cold items, 9 are outright wrong and 6 partial, so memory repairs both fundamentally-wrong ecosystem guesses and last-mile precision misses.

The paraphrase column’s single error is also the experiment’s one *regression*. Cold, p19 “run the benchmark suite” was answered `make benchmark`, which the judge accepted as a plausible route to the gold `python -m benchmarks.run`. With the paraphrased exemplar retrieved, the model instead answered `cargo bench`: it resolved the ecosystem to Rust, and the retriever did not surface the paraphrased Python exemplar strongly enough to override that prior. This is a retrieval-plus-prior miss, not a lookup failure, and it is the honest residual of a method that must generalize rather than copy; §5.4 reads it together with the Axis-B residuals.

A note on token cost. Memory’s effect on token cost is model-dependent, an observation that cuts against the claim we expected to make. On non-reasoning `claude-haiku-4-5`, retrieved exemplars add a small input cost (mean 95→161 tokens per call, +66). On a *reasoning* model (Qwen3-32B, from the free-tier runs of §5.1), the same few-shot grounding suppresses the verbose chain-of-thought the model otherwise emits when unsure, so net cost falls (385→217 per call, −44%).

A.3 Axis B: self-knowledge

Docs memory lifts overall accuracy from 43% to 76% (35→62 of 82), but the two question types

behave very differently. Retrieval turns a plausible guess into a grounded fact:

```
k02 "How do I save the current file?"  
none: "Use the Save action."  
      [would run: Save]          (acts, no key)  
docs: "Press C-x C-s to save  
       the current buffer."      (correct)
```

The reconfigure slice rises from 17% to 93% (5→28 of 30). Without docs the model invents plausible config locations: r04 “use a different model” points at a `~/mars/config.json` that does not exist. With the retrieved knob descriptions, which carry the exact name, file, and default, the agent answers `theme_accent / MARS_LLM_MODEL` in `tuning.json` correctly. The two failure families that motivated the corpus design separate cleanly here, which is how we apportion the caveat in Table 2: *acts-instead-of-explains* (answering a how-to question with a bare directive, as in k02 above) is repaired by the *explain-don’t-act* instruction, while *hallucinated identifiers* (a right-idea knob under the wrong filename) are repaired only by the retrieved, path-carrying knob descriptions. Knowledge questions gain far less (58% to 65%) because they turn on an *exact keybinding*, and even with the docs retrieved the model sometimes prefers a plausible chord over the one the docs state (k05 “detach my session” → C-x C-c, gold C-x C-d); §5.4 analyzes this knob-versus-chord asymmetry. A residual limit is judge slack: a few correct-but-differently-phrased answers are marked wrong, so the reported knowledge number is a mild lower bound even under the stronger `claude-sonnet-5` oracle.